Report on Assignment 1

OpenMP Homework

Students:

Shayan Alvansazyazdi
(5447411)
Sina Hatami
(5447389)
Behnoud Shafiezadeh Kenari
(5655740)

Feb 2024

pi_homework2.c

These lines include necessary header files for input/output operations (stdio.h), dynamic memory allocation (stdlib.h), time functions (time.h), mathematical functions (math.h), and OpenMP functions (omp.h).

```
#include "stdio.h" // printf
#include "stdlib.h" // malloc and rand for instance. Rand not thread safe!
#include "time.h"   // time(0) to get random seed
#include "math.h"   // sine and cosine
#include "omp.h"    // openmp library like timing
```

These lines define constants PI2 (twice the value of pi) and R_ERROR (a small value used for error comparison).

```
#define PI2 6.28318530718
#define R_ERROR 0.01
```

These lines declare prototypes for all the functions used in the code.

```
int DFT(int idft, double* xr, double* xi, double* Xr_o, double* Xi_o, int N);
int fillInput(double* xr, double* xi, int N);
int setOutputZero(double* Xr_o, double* Xi_o, int N);
int checkResults(double* xr, double* xi, double* xr_check, double* xi_check, double* Xr_o, double* Xi_r, int N);
int printResults(double* xr, double* xi, int N);
```

This is the entry point of the program. It takes command-line arguments argc and argv, although they are not used in this code.

```
int main(int argc, char* argv[]){
   // Main function body
}
```

This section declares variables such as `N` (size of input array 5000/10000/15000/20000) and dynamically allocates memory for arrays `xr`, `xi`, `xr_check`, `xi_check`, `Xr_o`, and `Xi_o`. The function `fillInput` initializes the input arrays `xr` and `xi`.

```
int N = 10000; // 5000,10000,15000,20000
printf("DFTW calculation with N = %d \n",N);
double* xr = (double*) malloc (N *sizeof(double));
double* xi = (double*) malloc (N *sizeof(double));
fillInput(xr,xi,N);
// Other variable declarations and memory allocations...
```

These lines compute the Discrete Fourier Transform (DFT) and its inverse (IDFT) using the `DFT` function.

```
int idft = 1;
DFT(idft,xr,xi,Xr_o,Xi_o,N);
idft = -1;
DFT(idft,Xr_o,Xi_o,xr_check,xi_check,N);
```

This section measures the execution time of specific code segments using `omp_get_wtime()` function.

```
double start_time = omp_get_wtime();
// Code section to be timed...
double run_time = omp_get_wtime() - start_time;
printf("DFTW computation in %f seconds\n",run_time);
```

These lines check the accuracy of DFT computation and display the results if debugging is enabled.

```
checkResults(xr,xi,xr_check,xi_check,Xr_o, Xi_o, N);
// Other code for result checking and output display...
```

This section deallocates the dynamically allocated memory and terminates the program.

```
free(xr); free(xi);
free(Xi_o); free(Xr_o);
free(xr_check); free(xi_check);
return 1;
```

This function computes the Discrete Fourier Transform (DFT) or its inverse (IDFT) based on the value of idft.
It takes input arrays xr and xi representing the real and imaginary parts of the input signal, respectively.
It calculates the DFT/IDFT and stores the results in the output arrays Xr_o and Xi_o.
The computation involves nested loops to iterate over all frequency components (k) and time samples (n).
The omp_set_num_threads(8) function sets the number of threads for parallelization to 8.
The #pragma omp parallel for private(k, n) directive parallelizes the outer loop, distributing the workload across multiple threads.
After the computation, if idft is -1 (indicating IDFT), it normalizes the output by dividing each element by N.

```
// DFT/IDFT routine
// idft: 1 direct DFT, -1 inverse IDFT (Inverse DFT)
int DFT(int idft, double* xr, double* xi, double* Xr_o, double* Xi_o, int N){
  int k, n;
  double start_time = omp_get_wtime();
  omp_set_num_threads(8); // set the number of threads to 8 for parallelization
  #pragma omp parallel for private(k, n) // parallelize the outer loop and declare private variables
  for (k = 0; k < N; k++) {
     for (n = 0; n < N; n++) {
        // Real part of X[k]
        Xr_o[k] += xr[n] * cos(n * k * PI2 / N) + idft * xi[n] * sin(n * k * PI2 / N);
        // Imaginary part of X[k]
        Xi_o[k] += -idft * xr[n] * sin(n * k * PI2 / N) + xi[n] * cos(n * k * PI2 / N);
     }
  }
```

This function initializes the input arrays xr and xi with a constant real signal and zeros for the imaginary part.
It iterates over the elements of the arrays and assigns the values accordingly.

```
// set the initial signal
// be careful with this
// rand() is NOT thread safe in case
int fillInput(double* xr, double* xi, int N){
  int n;
  // srand(time(0));
  // for(n=0; n < 100000;n++) // get some random number first
  //   rand();
  for(n=0; n < N;n++){
     // Generate random discrete-time signal x in range (-1,+1)
     //xr[n] = ((double)(2.0 * rand()) / RAND_MAX) - 1.0;
```

```
    //xi[n] = ((double)(2.0 * rand()) / RAND_MAX) - 1.0;
    // constant real signal
    xr[n] = 1.0;
    xi[n] = 0.0;
  }
  return 1;
}
```

This function sets the output arrays Xr_o and Xi_o to zero.
It iterates over the elements of the arrays and assigns zero to each element.

```
// set to zero the output vector
int setOutputZero(double* Xr_o, double* Xi_o, int N){
  int n;
  for(n=0; n < N;n++){
    Xr_o[n] = 0.0;
    Xi_o[n] = 0.0;
  }
  return 1;
}
```

This function checks the correctness of the DFT computation by comparing the computed results with the expected results.
It calculates the absolute difference between corresponding elements of the input and output arrays and checks if they are within a certain error threshold (R_ERROR).
If any discrepancy is found, it prints an error message indicating the indices and values of the mismatched elements.

```
// check if x = IDFT(DFT(x))
int checkResults(double* xr, double* xi, double* xr_check, double* xi_check, double* Xr_o, double* Xi_r, int N){
  int n;
  for(n=0; n < N;n++){
    if (fabs(xr[n] - xr_check[n]) > R_ERROR)
      printf("ERROR - x[%d] = %f, inv(X)[%d]=%f \n",n,xr[n], n,xr_check[n]);
      if (fabs(xi[n] - xi_check[n]) > R_ERROR)
      printf("ERROR - x[%d] = %f, inv(X)[%d]=%f \n",n,xi[n], n,xi_check[n]);

    }
    printf("Xre[0] = %f \n",Xr_o[0]);
    return 1;
}
```

This function prints the results of the DFT computation.
It iterates over the elements of the input arrays xr and xi and prints their values along with the corresponding indices.

```
// print the results of the DFT
int printResults(double* xr, double* xi, int N){
  int n;
  for(n=0; n < N;n++)
     printf("Xre[%d] = %f, Xim[%d] = %f \n", n, xr[n], n, xi[n]);
  return 1;
}
```

N	loop in Ft	normalizing Ft	loop in Ft	normalizing Ft	DFTW computation
5000	0.291524	0.000001	0.262463	0.000029	0.554099
10000	1.027627	0.000001	1.008378	0.000047	2.036131
15000	2.266709	0.000003	2.256068	0.000135	4.523019
20000	4.025246	0.000002	3.998218	0.000204	8.023793

Based on the provided outputs with different values of `N` :

1. Effect on Computation Time:
- As `N` increases, the computation time for the main loop of the Fourier transform (`Ftransform`) also increases. This is evident from the fact that for `N = 20000`, the computation time is approximately double that of `N = 10000`, and similarly, for `N = 15000`, it falls between the times for `N = 10000` and `N = 20000`.
- Conversely, decreasing `N` leads to shorter computation times. For example, for `N = 5000`, the computation time is significantly lower compared to larger values of `N`.

2. Effect on Total Computation Time:
- The total computation time (`DFTW computation`) is directly influenced by the main loop's computation time. As `N` increases, the total computation time also increases proportionally. For example, the total computation time roughly doubles when `N` is doubled from 10000 to 20000.

3. Effect on Normalization Time:

- The time taken for normalizing the Fourier transform (`time for normalizing Ftransform`) remains relatively constant regardless of the value of `N`. This suggests that the normalization process is not significantly impacted by changes in `N`.

4. Accuracy of Results:

- The real part of the first element (`Xre[0]`) in the output array remains consistent across different values of `N`, indicating that the accuracy of the computation is not compromised by changing `N`.

5. Conclusion:

- Increasing `N` results in longer computation times, while decreasing `N` leads to shorter computation times.

- The normalization time remains relatively constant regardless of `N`.

- Despite variations in computation time, the accuracy of the results, as indicated by `Xre[0]`, remains consistent.

- It's essential to choose an appropriate value of `N` based on the specific requirements of the application, considering the trade-off between accuracy and computational efficiency.